# Webové technológie 2

## 2. prednáška

Prof. Ing. Katarína Žáková, PhD.
katarina.zakova@stuba.sk

# Základné spracovanie formuláru

```php
<?php

$meno = $_GET['meno'];
$vek = $_GET['vek'];
echo $meno."<br>";
echo $vek."<br><br>";

$meno = $_REQUEST['meno'];
$vek = $_REQUEST['vek'];
echo $meno."<br>";
echo $vek."<br><br>";

?>
```

```php
<?php

$meno = $_POST['meno'];
$vek = $_POST['vek'];
echo $meno."<br>";
echo $vek."<br><br>";

$meno = $_REQUEST['meno'];
$vek = $_REQUEST['vek'];
echo $meno."<br>";
echo $vek."<br><br>";

?>
```

Meno: Martin Žák
Vek: 34
Odosli

Martin Žák
34

Martin Žák
34

# Nahrávanie súborov zo stránky

Pre odoslanie súboru na server slúži POST metóda a PHP funkcie na manipuláciu so súborom.

Na odoslanie súboru musíme vytvoriť formulár:

```
<form enctype="multipart/form-data" action="action_script.php" method="POST">
   <input type="hidden" name="MAX_FILE_SIZE" value="30000">
   Súbor:
   <input name="userfile" type="file">
   <input type="submit" value="Odoslať súbor">
</form>
```

action_script.php – je názov obslužného skriptu volaného po odoslaní formulára

MAX_FILE_SIZE – skryté pole určuje maximálnu veľkosť nahrávaného súboru.

# $_FILES

Informácie o nahrávanom súbore sú dostupné v premennej $_FILES

$_FILES['userfile']['name']

pôvodné meno nahrávaného súboru

$_FILES['userfile']['type']

MIME typ súboru

$_FILES['userfile']['size']

veľkosť súboru

$_FILES['userfile']['tmp_name']

dočasný súbor

$_FILES['userfile']['error']

chybová správa

# Upload súboru

```html
<html>

<body>
  <form action="FileUpload.php" method="post" enctype="multipart/form-data">
    <label for="file">Filename:</label>
    <input type="file" name="file" id="file">
    <br>
    <input type="submit" name="submit" value="Submit">
  </form>
</body>

</html>
```

Filename: E:\jia\gyurasz.pdf Browse...
Submit

```php
<?php
if ($_FILES["file"]["error"] > 0)
  { echo "Error: " . $_FILES["file"]["error"] . "<br>"; }
else
  { echo "Upload: " . $_FILES["file"]["name"] . "<br>";
    echo "Type: " . $_FILES["file"]["type"] . "<br>";
    echo "Size: " . ($_FILES["file"]["size"] / 1024) . " Kb<br>";
    echo "Stored in: " . $_FILES["file"]["tmp_name"] . "<br><br>";
  }
?>
```

Upload: gyurasz.pdf
Type: application/pdf
Size: 316.638671875 Kb
Stored in: /tmp/phpe6wxUa

Stored in: upload/gyurasz.pdf

```php
if (file_exists("upload/" . $_FILES["file"]["name"]))
  { echo $_FILES["file"]["name"] . " already exists. "; }
else
  { move_uploaded_file($_FILES["file"]["tmp_name"], "upload/" . $_FILES["file"]["name"]);
    echo "Stored in: " . "upload/" . $_FILES["file"]["name"];
  }
```

gyurasz.pdf already exists.

# Upload súboru

Spracovanie informácií o súbore v PHP

```
$uploaddir = '/var/www/uploads/';
$uploadfile = $uploaddir.basename($_FILES['userfile']['name']);

echo '<pre>';
if (move_uploaded_file($_FILES['userfile']['tmp_name'], $uploadfile))
  { echo "Súbor bol nahraný na server.\n";}
else
  { echo "Chyba!\n";}

echo 'Informácie o súbore:';
print_r($_FILES);

echo "</pre>";
```

# Nahrávanie súborov zo stránky

Chybové správy:

0 - UPLOAD_ERR_OK

nahratie súboru prebehlo úspešne

1 - UPLOAD_ERR_INI_SIZE

veľkosť súboru je väčšia ako definovaná v php.ini upload_max_filesize

2 - UPLOAD_ERR_FORM_SIZE

veľkosť súboru je väčšia ako definovaná v MAX_FILE_SIZE

3 - UPLOAD_ERR_PARTIAL

súbor bol nahratý iba čiastočne

4 - UPLOAD_ERR_NO_FILE

nebol nahraný žiaden súbor

6 - UPLOAD_ERR_NO_TMP_DIR

chýba adresár pre dočasné súbory

# Nastavenia v php.ini

Nastavenia pre nahrávanie súborov

```
835  ;;;;;;;;;;;;;;;;;
836  ; File Uploads ;
837  ;;;;;;;;;;;;;;;;;
838
839  ; Whether to allow HTTP file uploads.
840  ; https://php.net/file-uploads
841  file_uploads = On
842
843  ; Temporary directory for HTTP uploaded files (will use system default if not
844  ; specified).
845  ; https://php.net/upload-tmp-dir
846  ;upload_tmp_dir =
847
848  ; Maximum allowed size for uploaded files.
849  ; https://php.net/upload-max-filesize
850  upload_max_filesize = 2M
851
852  ; Maximum number of files that can be uploaded via a single request
853  max_file_uploads = 20
```

**file_uploads**

povoľuje, zakazuje nahrávania súborov, štandardne „1“

**upload_tmp_dir**

dočasný adresár pre ukladanie súborov pri nahrávaní.
Musí byť zapisovateľný akýmkoľvek používateľom, pod ktorým PHP beží.

**upload_max_filesize**

maximálna veľkosť nahrávaného súboru. Štandardne „2MB“

# Nahrávanie súborov zo stránky

Nahranie viac súborov:

```
<form action="" method="post" enctype="multipart/form-data">
  <p>Obrázky:
     <input type="file" name="pictures[]">
     <input type="file" name="pictures[]">
     <input type="file" name="pictures[]">
     <input type="submit" value="Send"> </p>
</form>
```

```
foreach ($_FILES["pictures"]["error"] as $key => $error)
 {if ($error == UPLOAD_ERR_OK)
   {$tmp_name = $_FILES["pictures"]["tmp_name"][$key];
    $name = $_FILES["pictures"]["name"][$key];
    move_uploaded_file($tmp_name, "data/$name");
   }
 }
```

# Práca s adresárom

```
1  <?
2  $path = "/home/kar/katarina/public_html/iia";
3  echo "Directory Listing of $path<br>";
4
5  $dir_handle = @opendir($path) or die("Unable to open $path");
6
7  while ($file = readdir($dir_handle))
8  {   echo "<a href='$file'>$file</a><br>"; }
9
10 closedir($dir_handle);
11 ?>
```

Directory Listing of /home/kar/katarina/public_html/iia

.
..
adresar.php
adresar_bez.php
FileForm.html
FileUpload.php
upload

Directory Listing of /home/kar/katarina/public_html/iia

adresar.php
adresar_bez.php
FileForm.html
FileUpload.php
upload

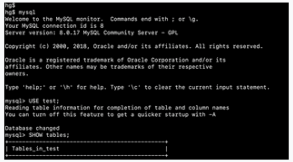
```
while ($file = readdir($dir_handle))
{   if($file!="." && $file!="..")
        echo "<a href='$file'>$file</a><br>";}
```

# Úvod k databázam

- pojem databáza: 60–te roky minulého storočia
- databázový model: hierarchický, sieťový, relačný
- databáza – tabuľka – stĺpce (dátový typ) – hodnota (užívateľské údaje)
- jazyk SQL: Structured Query Language
  - 70-te, 80-te roky minulého storočia
  - prvý štandard – 1986 – SQL86
  - nástroje pre tvorbu databáz, manipuláciu s údajmi
  - deklaratívny programovací jazyk
- MySQL
  - entitno-relačná databáza
  - asi najpoužívanejší voľne šíriteľný SQL-server (stabilný a rýchly)
  - prvá verzia 23.5.1995
  - komponent pre LAMP, WAMP, MAMP, LEMP, …

**MySQL**

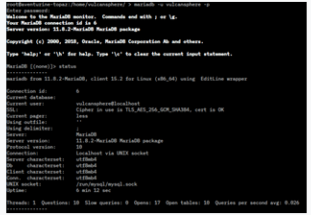
MySQL logo; the dolphin in the logo is named "Sakila".[1]



Screenshot of the default MySQL command line banner and prompt.

| | |
|---|---|
| **Original author** | MySQL AB |
| **Developer** | Oracle Corporation |
| **Initial release** | 23 May 1995; 30 years ago |
| **Stable release** | 9.5.0[2] / 21 October 2025; 3 months ago |
| **Written in** | C, C++[3] |
| **Operating system** | Linux, Solaris, macOS, Windows, FreeBSD[4] |
| **Available in** | English |
| **Type** | RDBMS |
| **License** | GPLv2 or proprietary[5] |
| **Website** | www.mysql.com |
| **Repository** | github.com/mysql/mysql-server |

**MariaDB**



| | |
|---|---|
| **Developers** | MariaDB plc, MariaDB Foundation |
| **Initial release** | 29 October 2009; 16 years ago[1] |
| **Stable release** | 12.1.2[2] / 18 November 2025; 2 months ago |
| **Written in** | C, C++, Perl, Bash |
| **Operating system** | Linux, Windows, macOS[3] |
| **Available in** | English |
| **Type** | RDBMS |
| **License** | GPLv2, LGPLv2.1 (client libraries)[4] |
| **Website** | mariadb.com (MariaDB plc) mariadb.org (MariaDB Foundation) |
| **Repository** | github.com/MariaDB/server |

# MySQL databáza

- pripojenie sa k MySQL serveru pomocou klienta mysql
```
mysql -u username -p
```

- vytvorenie databázy
```
CREATE DATABASE meno_databazy;
```
- pripojenie sa k databáze
```
USE meno_databazy;
```
- vymazanie databázy (aj s obsahom)
```
DROP DATABASE meno_databazy;
```

- zobrazenie zoznamu tabuliek
```
SHOW tables;
```
- zobrazenie štruktúry tabuľky
```
DESCRIBE meno_tabulky;
```
- vytvorenie tabuľky
```
CREATE TABLE osoby
  ( id INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
    meno VARCHAR(50) NOT NULL,
    priezvisko VARCHAR(50) NOT NULL,
    opis TINYTEXT,
    datum_narodenia DATE );
```
- vymazanie tabuľky
```
DROP TABLE meno_tabulky;
```

# Údajové typy v databázach

| DATE TYPE | SPEC | DATA TYPE | SPEC |
|---|---|---|---|
| CHAR | String (0 - 255) | INT | Integer (-2147483648 to 214748-3647) |
| VARCHAR | String (0 - 255) | BIGINT | Integer (-9223372036854775808 to 9223372036854775807) |
| TINYTEXT | String (0 - 255) | FLOAT | Decimal (precise to 23 digits) |
| TEXT | String (0 - 65535) | DOUBLE | Decimal (24 to 53 digits) |
| BLOB | String (0 - 65535) | DECIMAL | "DOUBLE" stored as string |
| MEDIUMTEXT | String (0 - 16777215) | DATE | YYYY-MM-DD |
| MEDIUMBLOB | String (0 - 16777215) | DATETIME | YYYY-MM-DD HH:MM:SS |
| LONGTEXT | String (0 - 4294967295) | TIMESTAMP | YYYYMMDDHHMMSS |
| LONGBLOB | String (0 - 4294967295) | TIME | HH:MM:SS |
| TINYINT | Integer (-128 to 127) | ENUM | One of preset options |
| SMALLINT | Integer (-32768 to 32767) | SET | Selection of preset options |
| MEDIUMINT | Integer (-8388608 to 8388607) | BOOLEAN | TINYINT(1) |

https://www.mysqltutorial.org/mysql-data-types.aspx

| Value | CHAR(4) | Storage Required |
|---|---|---|
| '' | '  ' | 4 bytes |
| 'ab' | 'ab  ' | 4 bytes |
| 'abcd' | 'abcd' | 4 bytes |
| 'abcdefgh' | 'abcd' | 4 bytes |

| Value | VARCHAR(4) | Storage Required |
|---|---|---|
| '' | '' | 1 byte |
| 'ab' | 'ab' | 3 bytes |
| 'abcd' | 'abcd' | 5 bytes |
| 'abcdefgh' | 'abcd' | 5 bytes |

**Návrh od AI:**

```
CREATE TABLE users (
    id INT,
    username VARCHAR(255),
    age INT,
    gender VARCHAR(255),
    bio VARCHAR(255)
);
```

# Výpis záznamov - SELECT

| id | meno | priezvisko | datum_narodenia |
|---|---|---|---|
| 1 | Peter | Nový | 1972-10-12 |
| 2 | Ján | Kováč | 1964-08-10 |
| 3 | Peter | Kováč | 1984-01-08 |
| 4 | Jozef | Hrach | 1978-12-18 |
| 5 | Anton | Marec | 1975-03-16 |

```
SELECT * FROM osoby;
```

```
SELECT * FROM osoby WHERE meno='Jozef';
```

| id | meno | priezvisko | datum_narodenia |
|---|---|---|---|
| 4 | Jozef | Hrach | 1978-12-18 |

```
SELECT * FROM osoby
  WHERE meno='Ján' OR meno='Jozef';
```

| id | meno | priezvisko | datum_narodenia |
|---|---|---|---|
| 2 | Ján | Kováč | 1964-08-10 |
| 4 | Jozef | Hrach | 1978-12-18 |

```
SELECT priezvisko FROM osoby
  WHERE meno='Peter';
```

| priezvisko |
|---|
| Nový |
| Kováč |

```
SELECT priezvisko, datum_narodenia FROM osoby
  WHERE meno='Peter';
```

| priezvisko | datum_narodenia |
|---|---|
| Nový | 1972-10-12 |
| Kováč | 1984-01-08 |

```
SELECT COUNT(*) FROM osoby;
```

| COUNT(*) |
|---|
| 5 |

```
SELECT * FROM osoby WHERE meno LIKE 'Peter';
```

| id | meno | priezvisko | datum_narodenia |
|---|---|---|---|
| 1 | Peter | Nový | 1972-10-12 |
| 3 | Peter | Kováč | 1984-01-08 |

```
SELECT meno, priezvisko
  FROM osoby
  WHERE meno LIKE 'j%';
```

| id | meno | priezvisko |
|---|---|---|
| 2 | Ján | Kováč |
| 4 | Jozef | Hrach |

```
SELECT meno, priezvisko
  FROM osoby
  WHERE meno LIKE '%n';
```

| id | meno | priezvisko |
|---|---|---|
| 2 | Ján | Kováč |
| 5 | Anton | Marec |

```
SELECT meno, priezvisko
  FROM osoby
  WHERE priezvisko
  LIKE '%ov%';
```

| id | meno | priezvisko |
|---|---|---|
| 1 | Peter | Nový |
| 2 | Ján | Kováč |
| 3 | Peter | Kováč |

```
SELECT * FROM osoby
  WHERE priezvisko='Kováč'
  LIMIT 1;
```

| id | meno | priezvisko | datum_narodenia |
|---|---|---|---|
| 2 | Ján | Kováč | 1964-08-10 |

```
SELECT * FROM osoby WHERE (id > 3) AND (meno LIKE 'j%');
SELECT * FROM osoby ORDER BY meno;
SELECT * FROM osoby ORDER BY meno DESC;
```

# Vstavané funkcie MySQL

## Agregačné funkcie jazyka SQL

count() - počet záznamov v dotaze

```
SELECT COUNT(mesto) FROM studenti
  WHERE vek>18;
SELECT COUNT(*) FROM studenti WHERE vek>18;
SELECT COUNT(DISTINCT mesto) FROM studenti
  WHERE vek>18;
```

max(), min() - vracia maximálnu resp. minimálnu hodnotu z poľa číselného typu

```
SELECT MAX(id) AS maxid FROM studenti;
```

avg() - vracia aritmetický priemer zo sady hodnôt

```
SELECT AVG(vek) FROM studenti;
```

sum() - vracia súčet hodnôt zo špecifikovaného poľa

```
SELECT SUM(cena) AS celkom FROM studenti;
```

## Aritmetické operácie

```
SELECT (pole1 + pole2) AS sucet FROM testy
SELECT (pole1 / pole2) AS vysledok FROM testy
```

## Manipulácia s textom

concat – vracia 1 reťazec, pričom kombinuje hodnoty dvoch reťazov

```
SELECT concat(meno," ",priezvisko) AS meno,
  adresa, mesto FROM pouzivatelia;
```

ltrim – odstráni začiatočné medzery

```
SELECT LTRIM("     Michal")
```

rtrim – odstráni koncové medzery

```
SELECT RTRIM("Michal     ")
```

trim – odstráni začiatočné a koncové medzery, resp. iné znaky

```
SELECT TRIM("     Michal        ")
SELECT TRIM(TRAILING ".xxx" FROM "Michal.xxx")
SELECT TRIM(LEADING "xxx" FROM "xxxMichal")
SELECT TRIM(BOTH "xxx" FROM "xxxMichalxxx")
```

substring – vracia reťazec od polohy danej druhým parametrom

```
SELECT SUBSTRING("Ahoj svet!",6)
```

locate – vyhľadá prvý reťazec v druhom od polohy danej tretím parametrom vracia číslo udávajúce polohu hľadaného reťazca

```
SELECT LOCATE ("svet","Ahoj svet!",1)
```

replace – nahradí jeden reťazcový výraz druhým

```
SELECT REPLACE("nazov.xxx","xxx","pdf")
```

reverse – prevráti reťazcový výraz

```
SELECT REVERSE("tak")
```

ucase – prevod textového výrazu na veľké písmená

```
SELECT UCASE("michal")
```

lcase – prevod textového výrazu na malé písmená

```
SELECT LCASE("MICHal")
```

# INSERT, DELETE, UPDATE

Naplnenie tabuľky

```
INSERT INTO osoby VALUES (0,'Peter','Nový','1972-10-12');
INSERT INTO osoby (meno, priezvisko, datum_narodenia) VALUES('…', '…', '…')
```

Vymazávanie záznamov

```
DELETE FROM osoby WHERE id=4;
DELETE FROM osoby WHERE meno='Peter';
```

Modifikácia záznamov

```
UPDATE osoby SET meno = "Matej" WHERE id = 2;
UPDATE osoby SET Vek = Vek + 2 WHERE Vek > 23;
```

# Spájanie tabuliek (INNER JOIN)

- 2 tabuľky:
  - tabuľka študentov - evidencia osobných údajov
  - tabuľka predmetov – evidencia registrovaných predmetov pre každého študenta
- spoločný stĺpec – id_student

Výber všetkých registrovaných predmetov pre študentov, ktorí majú registrovaný nejaký predmet v 3.roč.:

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st, predmety3 AS pr3
    WHERE st.id_student = pr3.id_student AND st.id_student = 8
```

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu                ANSI-92
    FROM studenti AS st JOIN predmety3 AS pr3
    ON st.id_student = pr3.id_student WHERE st.id_student=8
```

- relačné operátory: =, <, >, <=, >=, <>

# Spájanie tabuliek (LEFT JOIN)

Vytvorenie ľavého vonkajšieho spojenia:

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st, predmety3 AS pr3
    WHERE st.id_student *= pr3.id_student
```

ANSI-92

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st
    LEFT JOIN predmety3 AS pr3
      ON st.id_student = pr3.id_student
```

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st
    LEFT JOIN predmety3 AS pr3 USING id_student
```

# Spájanie tabuliek (JOIN príklad)

Zamestnanci:

| id_z | id_p | meno | |
|---|---|---|---|
| 1 | 1 | Matej | ... |
| 2 | 2 | Peter | ... |
| 3 | 3 | Vlado | ... |
| 4 | 6 | Martin | ... |

Pozície:

| id_p | pozicia |
|---|---|
| 1 | manazer |
| 2 | uctovnik |
| 3 | pravnik |
| 4 | technik |
| 5 | asistent |

INNER = všetci zamestnanci, ku ktorým je možné nájsť pracovnú pozíciu (3)
LEFT = všetci zamestnanci aj s neznámou pracovnou pozíciou (4)
RIGHT = všetky pracovné pozície aj bez zamestnancov (5)

# Spájanie tabuliek (NATURAL LEFT JOIN, CROSS JOIN)

## NATURAL LEFT JOIN

ako súčasť spojenia využije všetky zhodné polia oboch zdrojových tabuliek

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st
    NATURAL LEFT JOIN predmety3 AS pr3
```

## CROSS JOIN

vracia všetky záznamy z oboch zdrojových tabuliek vrátane všetkých kombinácií

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st, predmety3 AS pr3
```

```
SELECT st.meno, st.priezvisko, pr3.nazov_predmetu
    FROM studenti AS st
    CROSS JOIN predmety3 AS pr3
```

# DELETE CASCADE, UPDATE CASCADE

| | idBloku | nazovBloku | pocetPoschodi |
|---|---|---|---|
| Edit Copy Delete | 1 | A-blok | 8 |
| Edit Copy Delete | 2 | B-blok | 7 |
| Edit Copy Delete | 3 | C-blok | 7 |
| Edit Copy Delete | 4 | D-blok | 7 |
| Edit Copy Delete | 5 | E-blok | 8 |
| Edit Copy Delete | 6 | T-blok | 2 |

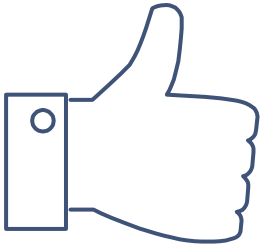
| | idUcebne | idBloku | nazovUcebne | velkost |
|---|---|---|---|---|
| Edit Copy Delete | 1 | 1 | A801 | 40 |
| Edit Copy Delete | 2 | 1 | A806 | 20 |
| Edit Copy Delete | 3 | 2 | B301 | 20 |
| Edit Copy Delete | 4 | 2 | B305 | 15 |
| Edit Copy Delete | 5 | 4 | D405 | 10 |
| Edit Copy Delete | 6 | 4 | D406 | 10 |
| Edit Copy Delete | 7 | 6 | C119 | 20 |
| Edit Copy Delete | 8 | 4 | D010 | 40 |

```
CREATE TABLE fakulta
( idBloku INT NOT NULL AUTO_INCREMENT,
  nazovBloku VARCHAR(10) NOT NULL,
  pocetPoschodi INT NOT NULL,
  PRIMARY KEY (idBloku)
) ENGINE=InnoDB;
```

```
CREATE TABLE ucebne
( idUcebne INT NOT NULL AUTO_INCREMENT,
  idBloku INT NOT NULL,
  nazovUcebne VARCHAR(10) NOT NULL,
  velkost INT,
  PRIMARY KEY (idUcebne)
) ENGINE=InnoDB;

ALTER TABLE ucebne
ADD CONSTRAINT fk_ucebne
    FOREIGN KEY (idBloku)
    REFERENCES fakulta(idBloku)
    ON UPDATE CASCADE
    ON DELETE CASCADE;
```

```
CREATE TABLE ucebne
( idUcebne INT NOT NULL AUTO_INCREMENT,
  idBloku INT NOT NULL,
  nazovUcebne VARCHAR(10) NOT NULL,
  velkost INT,
  PRIMARY KEY (idUcebne),
  CONSTRAINT fk_ucebne
    FOREIGN KEY (idBloku)
    REFERENCES fakulta(idBloku)
    ON UPDATE CASCADE
    ON DELETE CASCADE
) ENGINE=InnoDB;
```



# Ďakujem za pozornosť!

Nejaké otázky?